AI Interview Coach Technical Guide

Document version: 1.0 Generated on: 2026-03-14 Project root: `mastra-interview-coach`

---

1. Introduction

The AI Interview Coach application is a full-stack interview preparation system focused on frontend engineering roles. It helps a candidate practice technical interviews by:

- logging in with Google
- selecting a role, experience level, skill focus, and difficulty
- receiving a dynamically generated interview question
- answering in either text or code
- getting immediate feedback, scoring, and suggested next steps

The main problem this application solves is that interview preparation is usually static. Traditional question banks are repetitive, do not adapt to role seniority, and rarely explain why an answer is strong or weak. This application improves that by generating questions at runtime, avoiding duplicates, and scoring answers using a mix of heuristic evaluation and executable tests.

At a high level, the system combines:

- a React frontend for the user experience
- an Express backend for orchestration and APIs
- an LLM call to OpenAI for dynamic question generation
- a local reference retrieval system to steer topic selection
- an optional Judge0 integration for running coding answers

The result is an interview simulator that feels interactive, adaptive, and explainable.

---

2. High-Level System Architecture

2.1 Architecture Summary

The project is a monorepo with two apps:

1. `web`

A React + Vite frontend that handles login, setup, interview UI, and feedback rendering.

1. `server`

A TypeScript + Express backend that handles auth, sessions, question generation, evaluation, reference sync, and code execution.

There is also a light Mastra bootstrap layer, but most of the business logic lives in custom TypeScript code rather than a deep Mastra workflow graph.

2.2 Text Diagram

```
Browser
  -> React UI
  -> Axios API client
  -> Express API server
      -> Auth middleware
      -> InterviewAgent
          -> Reference retriever
          -> OpenAI Chat Completions API
          -> Fallback question generator
          -> Answer evaluator
          -> Judge0 or local JS VM
      -> Reference sync service
          -> Markdown files
          -> Web pages
          -> Static seed references
```

2.3 Frontend

The frontend is responsible for:

- loading the Google OAuth client
- restoring auth from local storage
- sending bearer tokens with API requests
- collecting interview profile input

- rendering the current question
- rendering answer input fields for theory or coding
- showing feedback, score, answer key, and debug data

The frontend does not generate questions or score answers. It is a thin orchestration layer over backend APIs.

2.4 Backend

The backend is responsible for:

- loading environment variables
- verifying Google access tokens
- creating lightweight interview sessions
- generating new interview questions
- evaluating user answers
- syncing and indexing reference resources
- executing coding solutions with Judge0 or a local JavaScript fallback

The backend is stateful in memory. Session data, recent questions, and evaluation history are stored in `Map` objects. This makes local development simple, but it also means state is lost when the server restarts.

2.5 AI / LLM Integration

The LLM integration is implemented manually with `axios` in `server/src/agents/interviewAgent.ts`. The backend:

- builds a strict system prompt
- builds a structured user payload
- calls OpenAI Chat Completions
- parses the JSON response
- sanitizes it
- validates its shape
- rejects weak or repeated questions

If the LLM is unavailable, the app falls back to a deterministic built-in question generator.

2.6 Data Flow

Here is the main question flow in text form:

```
User logs in
  -> frontend stores access token
  -> backend verifies token

User selects role + skill + difficulty
  -> frontend creates session

Frontend requests question
  -> backend resolves role context
  -> backend picks skill track and topic hints
  -> backend retrieves relevant reference snippets
  -> backend asks OpenAI for a question
  -> backend parses and validates result
  -> backend rejects duplicates or weak outputs
  -> backend returns final question

User submits answer
  -> backend evaluates answer
  -> backend returns score + correction + next steps
```

2.7 APIs

Main API routes:

- `GET /api/auth/me`
- `POST /api/session`
- `POST /api/question`
- `POST /api/answer`
- `POST /api/run`
- `GET /api/references/status`
- `POST /api/references/sync`

2.8 External Services

The application currently depends on these external systems:

- Google OAuth and Google token info endpoints
- OpenAI Chat Completions API
- Judge0, if configured

It also reads optional local markdown files and fetches external web pages for reference syncing.

---

3. Application Creation Flow (Step-by-Step)

This section explains how the app is built up conceptually from the ground up.

3.1 Project Initialization

The project starts as a monorepo with separate `server` and `web` packages.

Why this is necessary:

- frontend and backend have different runtimes and dependencies
- each side can be built and run independently
- local development is easier because both apps can start together from the root

3.2 Environment Configuration

The server loads `.env` and `.env.local` files from several candidate directories. Later-loaded `.env.local` files override earlier values.

Important environment variables include:

- `OPENAI_API_KEY`
- `QUESTION_GEN_MODEL`
- `GOOGLE_OAUTH_CLIENT_ID`
- `AUTH_DISABLED`
- `JUDGE0_BASE_URL`
- `JUDGE0_API_KEY`
- `JUDGE0_API_HOST`
- `JUDGE0_LOCAL_FALLBACK`
- `REFERENCE_SYNC_WEB_URLS`
- `INTERVIEW_RESOURCES_MD_PATH`
- `INTERVIEW_RESOURCES_MD_PATHS`
- `REFERENCE_SYNC_INTERVAL_MIN`

Why this is necessary:

- secrets cannot live in source code
- model selection should be configurable
- external services differ between local and production environments

3.3 Data Models

The core data model is built around:

- interview profile

- role bands
- question type
- question payload
- evaluation result
- reference documents

Why this is necessary:

- the frontend and backend need stable shapes to exchange data
- the LLM response must be normalized into a predictable structure
- the evaluator needs a known format for tests and answer keys

3.4 Reference Resources

The app begins with static seed references in `interviewReferences.ts`, then optionally expands them through auto-sync.

Why this is necessary:

- reference content gives the system domain context
- it helps question generation stay grounded in frontend, React, JavaScript, and Adobe Commerce topics
- it helps the backend retrieve focused snippets instead of sending the whole corpus to the model

3.5 Reference Sync Layer

The sync layer can read:

- markdown files
- web pages
- static seed references

The code also contains PDF extraction helpers, but the current sync path does not wire them into the active retriever pipeline.

Why this is necessary:

- static references alone go stale
- syncing lets the backend refresh context without redeploying the app

3.6 Retrieval Layer

The retriever tokenizes the query, scores reference documents by tag and text match, and returns the top hits.

Why this is necessary:

- the model does better with a small amount of targeted context
- retrieval reduces prompt size compared with injecting every document

3.7 AI Prompt Engineering

The backend creates:

- a system prompt that defines strict behavior rules
- a user payload that contains role, skills, recent topics, recent signatures, retrieved references, and required schema

Why this is necessary:

- raw prompting gives inconsistent output
- structured prompting improves JSON validity and question quality
- the prompt needs memory of recent questions to avoid repetition

3.8 API Integration

The frontend calls backend APIs using Axios. The backend then calls Google, OpenAI, and optionally Judge0.

Why this is necessary:

- the frontend should never call OpenAI directly with private API keys
- the backend is the correct place to enforce validation, retries, and fallback logic

3.9 Question Generation Pipeline

When a question is requested, the backend:

1. resolves role context
2. picks a skill track
3. selects topic hints
4. retrieves reference snippets
5. builds the prompt
6. calls the LLM
7. parses JSON
8. validates and sanitizes the payload
9. rejects duplicates or weak content
10. falls back to built-in question generation if needed

Why this is necessary:

- LLM output is probabilistic
- generation must be controlled, checked, and recoverable

3.10 Response Parsing and Validation

The backend uses helper functions to:

- extract the JSON object from raw model text
- sanitize hints and answers
- enforce valid coding function names
- make sure coding tests exist

Why this is necessary:

- models often add extra prose
- unsafe or incomplete outputs would break the UI and evaluator

3.11 Storage and Caching

The system uses lightweight in-memory maps for:

- sessions
- question history
- recent signatures
- recent topics
- evaluation results
- Google token cache

Why this is necessary:

- caching reduces duplicate work and repeated Google verification calls
- recent-question memory is required to suppress repetition

3.12 UI Rendering

Finally, the frontend renders:

- login UI
- profile setup UI
- question panel
- answer input
- code runner
- debug panel
- feedback and scoring

Why this is necessary:

- the same backend can power multiple question types, but the UI must present each mode clearly

---

4. Folder Structure and Project Organization

4.1 Root Structure

```
mastra-interview-coach/
  docs/
  server/
    src/
      agents/
      resources/
      tools/
      workflows/
    dist/
  web/
    src/
      components/
      services/
      types/
```

4.2 Folder Responsibilities

`docs/`

Stores architecture and developer documentation.

`server/src/agents/`

Contains the main interview orchestration logic. `interviewAgent.ts` is the heart of question generation and answer evaluation.

`server/src/resources/`

Contains the reference corpus and sync code.

- `interviewReferences.ts` : static seed documents
- `referenceSync.ts` : auto-sync from markdown and web sources

`server/src/tools/`

Contains utility services used by the agent.

- `ragRetriever.ts` : keyword-based reference retrieval
- `judgeRunner.ts` : code execution through Judge0 or local VM

`server/src/workflows/`

Contains workflow-related files. In the current codebase, `interviewFlow.ts` is empty, so runtime flow is implemented directly in `InterviewAgent`.

`server/src/index.ts`

Backend entrypoint. It loads env vars, sets up Express, registers routes, and starts the server.

`server/src/mastra.ts`

Creates a lightweight Mastra-style server bootstrap. It sets up session creation, registers `InterviewAgent`, and starts reference auto-sync.

`web/src/components/`

Contains the user interface:

- `LoginPage.tsx`
- `ProfileSetupPage.tsx`
- `InterviewUI.tsx`
- `VoiceControls.tsx`

`web/src/services/`

Contains API clients. `api.ts` is the frontend bridge to backend endpoints.

`web/src/types/`

Contains shared frontend type definitions for profile setup and role options.

4.3 How Files Interact

```
App.tsx
  -> LoginPage.tsx
  -> ProfileSetupPage.tsx
  -> InterviewUI.tsx
      -> api.ts
          -> server/src/index.ts routes
              -> createMastraServer()
                  -> InterviewAgent
                      -> ragRetriever
                      -> judgeRunner
                      -> referenceSync
```

---

5. Detailed Code Walkthrough

This section explains the most important classes, methods, and functions.

5.1 Server Bootstrap and HTTP Layer

`loadEnvFiles()`

- What it does: loads `.env` and `.env.local` files from candidate directories.
- Inputs: none.
- Output: none.
- Why it exists: the server needs a predictable way to resolve config.
- Connection: runs before Express starts, so later code can read `process.env`.

`verifyGoogleAccessToken(accessToken)`

- What it does: checks whether a Google access token is valid by calling Google's token info endpoint.
- Inputs: a bearer token string.
- Output: Google token info or `null`.
- Why it exists: backend routes are protected and must trust the caller.
- Connection: called by `requireAuth`.

`requireAuth(req, res, next)`

- What it does: protects API routes unless `AUTH_DISABLED=true`.
- Inputs: Express request/response/next objects.
- Output: either calls `next()` or returns `401`.

- Why it exists: prevents unauthorized access to question generation and evaluation.
- Connection: used by every protected route in `index.ts`.

`createMastraServer()`

- What it does: creates a simple session store, registers `InterviewAgent`, and starts reference auto-sync.
- Inputs: none.
- Output: `{ mastra, agentRegistry }`.
- Why it exists: gives the server a single bootstrap point.
- Connection: called once in `index.ts`.

Important note:

The current `InterviewAgent` constructor does not use the `mastra` or `sessions` arguments it receives. Session memory is actually stored inside the agent's own `stateBySession` map.

5.2 API Route Handlers

`POST /api/session`

- What it does: creates a new lightweight session id.
- Inputs: `profile`.
- Output: `{ ok, sessionId, profile }`.
- Why it exists: the frontend needs a session key before asking for questions.
- Connection: called when `InterviewUI` mounts.

`POST /api/question`

- What it does: asks `InterviewAgent` for the next question.
- Inputs: `sessionId`, `level`, `profile`.
- Output: question payload.
- Why it exists: this is the main entrypoint for runtime question generation.
- Connection: frontend calls it for the first question and every next question.

`POST /api/answer`

- What it does: sends a theory or coding answer to the agent for evaluation.
- Inputs: `sessionId`, `questionId`, `answerText`.
- Output: evaluation result.
- Why it exists: centralizes scoring and correction logic on the server.

- Connection: called after the user clicks "Evaluate Answer" or "Evaluate Code".

`POST /api/run`

- What it does: executes user code.
- Inputs: `language`, `source`, `stdin`.
- Output: Judge0 or local VM execution result.
- Why it exists: lets users test code before final evaluation.
- Connection: used by the code runner panel in the frontend.

5.3 Core InterviewAgent Utilities

`toDifficulty(level)`

- What it does: normalizes the requested difficulty.
- Inputs: raw string.
- Output: `easy`, `medium`, or `hard`.
- Why it exists: protects the pipeline from invalid values.
- Connection: used at the start of question generation.

`resolveRoleContext(profile)`

- What it does: derives role band and years of experience from the profile.
- Inputs: session profile.
- Output: `{ roleBand, yearsExperience }`.
- Why it exists: question difficulty and tone depend on seniority.
- Connection: used by `nextQuestionHandler`.

`choosePreferredQuestionType(difficulty, yearsExperience)`

- What it does: picks theory vs coding using weighted randomness.
- Inputs: difficulty and years of experience.
- Output: `theory` or `coding`.
- Why it exists: the app should not be locked into only one question type.
- Connection: seeds the LLM and fallback generation strategy.

`signatureForQuestion(title, questionText)`

- What it does: creates a normalized fingerprint of a question.
- Inputs: title and question text.
- Output: short normalized string.
- Why it exists: helps block repeated questions.
- Connection: stored in per-session history.

`isRepeatedSignature(signature, recentSignatures)`

- What it does: checks whether a question is too similar to earlier ones.
- Inputs: current signature and recent signatures.
- Output: boolean.
- Why it exists: duplicate prevention.
- Connection: used after every generated candidate.

Implementation detail:

- exact match is rejected
- semantic similarity is approximated with Jaccard similarity on token sets
- the threshold is `0.56`

`isRepeatedTopic(topic, recentTopics)`

- What it does: blocks repeated topic focus.
- Inputs: normalized topic and recent topics.
- Output: boolean.
- Why it exists: even if wording changes, the same concept should not repeat too often.
- Connection: used with signature filtering.

Implementation detail:

- topic similarity threshold is `0.7`

5.4 LLM Prompt Construction and Parsing

`generateQuestionWithLLM(input)`

- What it does: builds the OpenAI request, calls the API, extracts JSON, and normalizes the result.
- Inputs:
- difficulty
- preferredType
- roleContext
- skills
- focusSkill
- topicHints
- recentTopics
- mustIncludeEds

- recentSignatures
- resourceContext
- Output: `GeneratedQuestionPayload`
- Why it exists: this is the AI generation boundary.
- Connection: called from `nextQuestionHandler`.

Key steps inside the function:

```
const response = await axios.post(
  "https://api.openai.com/v1/chat/completions",
  {
    model,
    temperature: 0.35,
    messages: [
      { role: "system", content: system },
      { role: "user", content: JSON.stringify(userPayload) },
    ],
  }
);
```

Important implementation note:

The current code defaults `QUESTION_GEN_MODEL` to `gpt-4.1`. The README and older docs still mention `gpt-4o-mini`, so the code is the real source of truth.

`extractJsonObject(text)`

- What it does: finds the first `{` and last `}` and returns that slice.
- Inputs: raw model text.
- Output: JSON-looking substring.
- Why it exists: models sometimes add stray text around JSON.
- Connection: called before `JSON.parse`.

`coerceGeneratedPayload(raw, preferredType)`

- What it does: sanitizes and normalizes raw model output.
- Inputs: parsed object and preferred type.
- Output: safe `GeneratedQuestionPayload`.
- Why it exists: the model cannot be trusted to follow the schema perfectly.
- Connection: final step before converting to an interview question.

Validation rules include:

- `type` must be `theory` or `coding`
- hints are cleaned and clipped
- noise about references and PDFs is removed
- coding function names must be valid identifiers
- coding tests must contain at least 2 usable entries

`questionPayloadToInterviewQuestion(payload, opts)`

- What it does: converts the generated payload into the frontend-facing question object.
- Inputs: normalized payload and `mustIncludeEds`.
- Output: `InterviewQuestion`.
- Why it exists: adds ids and converts coding tests into runtime test metadata.
- Connection: used by both LLM output and fallback output.

5.5 Fallback Generation and Quality Control

`buildEmergencyDynamicQuestion(input)`

- What it does: produces a deterministic question when LLM generation fails or is rejected.
- Inputs:
- difficulty
- preferredType
- roleContext
- skills
- mustIncludeEds
- topicHints
- avoidTopics
- Output: `GeneratedQuestionPayload`
- Why it exists: the product must keep working without OpenAI.
- Connection: used after LLM retries fail.

This function contains:

- topic-based theory fallback generation
- built-in coding variants grouped by skill track and difficulty
- ready-made test cases for coding tasks

`buildConcreteTheoryAnswer(input)`

- What it does: creates a strong concrete answer for many known topics.
- Inputs: topic, skill track, and Adobe Commerce flag.
- Output: a detailed canonical answer string.
- Why it exists: LLM theory answers are often too vague or too meta.
- Connection: used in fallback mode and in LLM answer normalization.

`isWeakTheoryAnswer(answer)`

- What it does: detects meta answers like "a strong answer should..."
- Inputs: answer text.
- Output: boolean.
- Why it exists: canonical answers must answer the question, not describe how to answer.
- Connection: used after generation.

`isToyCodingQuestion(question, difficulty, roleContext)`

- What it does: rejects trivial coding prompts for hard and senior profiles.
- Inputs: question, difficulty, role context.
- Output: boolean.
- Why it exists: senior candidates should not receive toy prompts.
- Connection: applied to LLM and fallback candidates.

5.6 Main Question Generation Handler

`nextQuestionHandler(input)`

- What it does: orchestrates the full question-generation process.
- Inputs: session id, profile, level.
- Output: `InterviewQuestion`.
- Why it exists: it is the main backend engine behind `/api/question`.
- Connection: called by `InterviewAgent.call()`.

What it does in order:

1. resolve session and role context
2. pick preferred question type
3. detect active skill track
4. select topic hints
5. load recent signatures and topics

6. retrieve reference hits
7. attempt LLM generation with retries and type switching
8. reject duplicates, weak answers, or toy coding tasks
9. fall back to deterministic generation if needed
10. store debug info and remember topic/signature

This is the single most important runtime function in the application.

5.7 Answer Evaluation

`evaluateAnswerHandler(input)`

- What it does: scores theory answers or coding answers.
- Inputs: session id, question id, answer text.
- Output: `EvaluationResult`.
- Why it exists: feedback must align with the exact question shown to the user.
- Connection: main backend engine behind `/api/answer`.

For theory answers:

- it scores keyword coverage against the canonical answer
- it generates simple qualitative feedback
- it returns the canonical answer as the correction

For coding answers:

- it builds a coding harness
- executes the candidate code
- parses test output
- computes a score
- returns test-by-test pass/fail data

`buildCodingHarness(source, codingSpec)`

- What it does: wraps user code in a mini test runner.
- Inputs: raw source code and coding spec.
- Output: executable JavaScript string.
- Why it exists: candidate functions need to be run consistently.
- Connection: used in coding evaluation before `runCode()`.

The harness prints a special marker:

```
const marker = "__RESULT__:";
```

This lets the backend reliably find structured test results inside raw stdout.

`parseHarnessResults(stdout)`

- What it does: reads structured harness output from stdout.
- Inputs: stdout string.
- Output: parsed test result array or `null`.
- Why it exists: coding evaluation needs machine-readable pass/fail data.
- Connection: used after execution.

`runCode(input)` in `judgeRunner.ts`

- What it does: executes code through Judge0 or a local Node VM fallback.
- Inputs: language, source, stdin.
- Output: execution result object.
- Why it exists: code evaluation must run actual code, not just read text.
- Connection: called from `/api/run` and `evaluateAnswerHandler`.

5.8 Reference Retrieval and Sync

`retrieveReferenceHits(input)` in `ragRetriever.ts`

- What it does: returns top matching reference snippets using keyword scoring.
- Inputs: query and optional `topK`.
- Output: array of `RetrieverHit`.
- Why it exists: gives the question generator focused context.
- Connection: used inside `nextQuestionHandler`.

Important note:

This is not embedding-based retrieval. It is a keyword scorer over tags and text.

`setReferenceDocs(nextDocs)`

- What it does: replaces the active runtime reference set.
- Inputs: reference documents.
- Output: none.
- Why it exists: reference auto-sync needs a way to refresh the retriever.
- Connection: called by `syncInterviewReferences`.

`buildMarkdownDocs(mdPath)`

- What it does: reads markdown, strips markdown syntax, chunks text, and builds `ReferenceDoc` objects.
- Inputs: markdown file path.
- Output: docs or warning.
- Why it exists: local markdown should become searchable runtime context.
- Connection: used by sync.

`buildWebDocs(url)`

- What it does: downloads HTML, strips tags, chunks text, and builds `ReferenceDoc` objects.
- Inputs: URL.
- Output: searchable web docs.
- Why it exists: lets the app ingest web-based reference material.
- Connection: used by sync.

`syncInterviewReferences(reason)`

- What it does: refreshes runtime reference docs from markdown and web sources.
- Inputs: sync reason (`startup`, `interval`, `manual_api`).
- Output: sync status.
- Why it exists: keeps the reference layer current.
- Connection: called at startup, by timer, and by API.

Important implementation note:

The code contains PDF parsing helpers such as `buildPdfDocs()`, but the current sync flow does not call them, and the retriever filters out `pdf` docs. In the current runtime path, web and markdown documents are the active reference sources.

5.9 Frontend Components

`App()`

- What it does: controls the top-level app stages.
- Inputs: none.
- Output: React UI tree.
- Why it exists: the app moves through login -> profile setup -> interview.
- Connection: root component rendered from `main.tsx`.

`LoginPage`

- What it does: loads Google OAuth script and gets an access token.
- Inputs: `onSuccess`.
- Output: login UI and auth callback.
- Why it exists: protected backend routes need a verified identity.
- Connection: used by `App`.

`ProfileSetupPage`

- What it does: collects role, years, active skill, and difficulty.
- Inputs: `onStart`.
- Output: interview setup payload.
- Why it exists: question generation depends on profile context.
- Connection: used by `App`, then consumed by `InterviewUI`.

`InterviewUI`

- What it does: creates a session, requests questions, submits answers, runs code, and renders feedback.
- Inputs: profile, level, back callback.
- Output: the main interview screen.
- Why it exists: this is the user-facing runtime experience.
- Connection: talks to every backend route except auth.

Useful behavior implemented here:

- automatically creates a session on mount
- automatically fetches the first question
- prefetches the next question after evaluation
- supports answer key reveal
- shows a debug panel for generated questions

`VoiceControls`

- What it does: uses browser speech recognition to append spoken text into the theory answer box.
- Inputs: `onResult`.
- Output: voice-control button and transcript callback.
- Why it exists: improves accessibility and interview simulation.
- Connection: used only for theory answers.

---

6. AI Prompt Engineering Design

6.1 How the System Prompt Works

The system prompt acts like a rules contract for the model. It tells the model:

- return only JSON
- generate one question at a time
- avoid mentioning source documents
- avoid role names in question text
- avoid answer-writing advice
- keep coding tasks pure and testable
- keep senior hard prompts non-trivial
- respect Adobe Commerce / Preact constraints when needed

This matters because the model is probabilistic. Without strict instructions, it may:

- return markdown
- add commentary around the JSON
- produce vague answers
- create coding tasks that cannot be auto-tested

6.2 How the User Payload Is Structured

The payload contains the dynamic context for the current request:

```
const userPayload = {
  difficulty,
  preferredType,
  roleBand,
  yearsExperience,
  focusSkill,
  topicHints,
  recentCoreTopicsToAvoid,
  skills,
  mustIncludeEds,
  recentQuestionSignaturesToAvoid,
  referenceContext,
  requiredSchema
};
```

This is a strong design because it separates:

- permanent rules in the system prompt
- request-specific data in the user payload

6.3 How the LLM Generates Questions

The model receives:

- who the question is for
- which skill track to focus on
- which topics are preferred
- which topics and signatures must be avoided
- which references are relevant
- the exact output schema

This lets the model produce a question that is:

- relevant
- structured
- difficult enough
- less repetitive

6.4 Why the Constraints Exist

Some constraints look strict, but they solve real product problems:

- "JSON only" avoids parser failures
- "no role names in question text" avoids awkward wording
- "no references or PDFs in question text" hides internal system details from the user
- "no classes, no async, no DOM" keeps coding tasks executable in a simple harness
- "use one topic from topicHints" keeps questions focused

6.5 How Duplicate Questions Are Prevented

The system uses multiple defenses:

1. recent signatures are stored per track
2. recent normalized topics are stored per track
3. the LLM is explicitly told not to repeat them
4. generated results are checked locally after generation
5. Jaccard similarity thresholds reject semantically similar questions

This multi-layer design is important. Prompt instructions alone are not enough.

---

7. Question Generation Pipeline

This is the full runtime flow when a user requests a question.

```
User Request
  -> Build role context
  -> Detect skill track
  -> Pick topic hints
  -> Retrieve reference snippets
  -> Build LLM prompt
  -> Send request to OpenAI
  -> Receive raw response
  -> Extract JSON
  -> Normalize payload
  -> Reject duplicates / weak outputs
  -> Convert to InterviewQuestion
  -> Store debug info and recent history
  -> Return question
```

7.1 User Request

The frontend calls `/api/question` with:

- `sessionId`
- `level`
- `profile`

7.2 Context Resolution

The backend calculates:

- normalized difficulty
- role band
- years of experience
- active skill
- skill track

This turns loosely entered profile data into deterministic internal values.

7.3 Topic Selection

The backend chooses a topic list from `CORE_TOPIC_HINTS_BY_TRACK`. It shuffles topics and removes recently used ones where possible.

7.4 Reference Retrieval

The backend builds a query string from:

- difficulty
- role band
- years
- skills
- top topic hints
- a fixed "core fundamentals interview questions" suffix

Then it retrieves the top 3 matching snippets.

7.5 Prompt Build

The backend combines:

- strict system instructions
- structured user payload

7.6 OpenAI Call

The backend calls Chat Completions with:

- selected model
- prompt messages
- temperature `0.35`
- 30-second timeout

7.7 Response Parsing

The backend:

- reads `response.data.choices[0].message.content`
- flattens content arrays if needed
- extracts the JSON object
- parses it

7.8 Validation and Sanitization

The backend removes:

- reference noise
- malformed hints
- invalid coding specs

It also restores `preferredType` if the model returns an invalid `type`.

7.9 Quality Rejection

Candidates are rejected if:

- question text is missing
- canonical answer is missing
- theory answer is weak or meta
- coding task is too trivial
- signature is repeated
- topic is repeated

7.10 Fallback

If LLM attempts fail, the backend uses `buildEmergencyDynamicQuestion()`.

This means the product still works even when:

- OpenAI is unavailable
- the API key is missing
- the model returns invalid JSON
- all generated candidates are rejected

7.11 Final Response

The question is stored in session state, annotated with debug info, and returned to the frontend.

---

8. Reference Resource System

8.1 `ReferenceDoc` Structure

Every reference document follows this shape:

```
type ReferenceDoc = {
  id: string;
  sourceType: "pdf" | "web" | "md";
  title: string;
  sourceLabel: string;
  url?: string;
  tags: string[];
  content: string;
};
```

8.2 Static Reference Seeds

`interviewReferences.ts` contains hand-written seed documents for:

- JavaScript
- React
- Preact
- frontend system design
- Adobe Commerce storefront setup
- Adobe Commerce Drop-ins

These seeds provide a curated baseline corpus even before auto-sync runs.

8.3 Tagging System

Each reference document includes tags such as:

- `javascript`
- `hooks`
- `performance`
- `drop-ins`
- `system-design`

The retriever uses tags as a strong scoring signal. A token match in tags scores higher than a plain text match.

8.4 Auto-Sync Sources

The sync system can ingest:

- markdown files from disk
- web pages from static or env-provided URLs

The code also contains PDF extraction utilities, but they are not currently active in the runtime retrieval path.

8.5 How References Influence Question Generation

References influence generation indirectly, not as a full RAG answer system.

The flow is:

1. retrieve top reference hits
2. convert them into short `title: excerpt` strings
3. pass them in `referenceContext` to the LLM
4. tell the model not to mention the references explicitly

This design gives the model guidance without exposing internal source material to the user.

8.6 Important Current Limitation

In the current implementation:

- the retriever filters out `pdf` documents
- `syncInterviewReferences()` syncs markdown and web documents only
- `buildPdfDocs()` exists but is not wired into the sync pipeline

So, in practical runtime terms, web and markdown references are the active sources that influence question generation.

---

9. Error Handling and Edge Cases

9.1 Invalid JSON from the LLM

Handled by:

- `extractJsonObject()`
- `JSON.parse(...)`
- retry loop in `nextQuestionHandler`
- final fallback generator

If parsing fails, the question request does not crash the whole app. The backend records a debug note and tries again or falls back.

9.2 LLM Hallucinations or Weak Outputs

Handled by:

- strict prompt rules
- `coerceGeneratedPayload()`
- `stripReferenceNoise()`
- `isWeakTheoryAnswer()`
- `isToyCodingQuestion()`

This is important because models often produce valid-looking but low-value content.

9.3 Duplicate Questions

Handled by:

- recent signature tracking
- topic tracking
- Jaccard similarity checks
- retry loop

9.4 API Timeouts

Current timeouts include:

- Google token verification: 8 seconds
- OpenAI call: 30 seconds
- reference web fetch: 15 seconds
- local JavaScript execution: 3 seconds

These limits stop the app from hanging forever.

9.5 Missing Environment Variables

Behavior today:

- missing `OPENAI_API_KEY`: LLM generation is skipped and fallback mode is used
- missing Google client id on frontend: login fails with a UI error
- missing Judge0 config: local JS fallback is used for JavaScript where allowed
- missing markdown paths: sync continues without markdown docs

9.6 Server Restarts

Because storage is in memory:

- session ids are lost
- recent question memory is lost
- evaluation history is lost

This is acceptable for a local demo, but not for durable production sessions.

9.7 Reference Sync Failures

Sync errors are collected into `ReferenceSyncStatus.errors` instead of crashing the app.

This is a good design because the interview flow can continue even if sync is partially broken.

---

10. Performance and Cost Optimization

10.1 Prompt Compression

The system already reduces prompt size by:

- using only the top 3 reference hits
- clipping excerpts
- sending short topic hints instead of raw documents
- requesting JSON only

This saves tokens and improves response predictability.

10.2 Caching

Current caching strategies:

- Google access token cache in memory
- session question history in memory
- recent topic/signature memory in memory
- frontend auth restoration from local storage
- next-question prefetch in the frontend

These are lightweight but effective for a demo-scale app.

10.3 Code Execution Cost Control

Coding runs use:

- Judge0 when configured

- local Node VM fallback for JavaScript when not configured or unauthorized

This lowers dependency on external execution infrastructure during development.

10.4 Model Selection

The current code reads `QUESTION_GEN_MODEL` and falls back to `gpt-4.1`.

This is a useful optimization hook because developers can choose:

- cheaper models for local testing
- stronger models for production or higher-quality generation

10.5 Embeddings

The system does not currently use embeddings.

That means:

- retrieval is fast and simple
- setup is easy
- semantic matching is limited

10.6 Similarity Detection

The app uses normalized signatures and Jaccard similarity instead of embeddings.

This is cheaper than vector similarity, but it is also less semantically aware.

---

11. Future Improvements

Here are the most valuable next steps for the system.

11.1 Add Persistent Storage

Replace in-memory maps with a database or cache store such as:

- Postgres
- Redis
- SQLite for local mode

Benefits:

- durable sessions
- audit trails
- analytics

- multi-instance support

11.2 Use Embeddings for Retrieval and Duplicate Detection

Use vector embeddings for:

- semantic retrieval over references
- semantic duplicate-question detection
- answer similarity scoring

Benefits:

- more accurate retrieval
- better duplicate suppression
- less dependence on keyword overlap

11.3 Adaptive Difficulty

Adjust future questions based on:

- previous score
- answer length
- coding test pass rate
- repeated weak spots

Benefits:

- more personalized interview coaching
- stronger progression over time

11.4 Real Question Bank Persistence

Store generated and fallback questions in a searchable database.

Benefits:

- analytics on coverage
- caching of good questions
- review of rejected or weak generations

11.5 Better Evaluation Scoring

Current theory scoring is keyword-based. This can be improved by:

- rubric-based LLM evaluation
- structured answer decomposition
- topic-specific scoring templates

Current coding scoring can be improved by:

- hidden test cases
- complexity analysis
- style and correctness checks

11.6 Wire PDF Sync into Active Retrieval

The code already contains PDF extraction helpers. A future improvement would:

- activate `buildPdfDocs()` in `syncInterviewReferences()`
- stop filtering PDF docs out of the active retriever

11.7 Stronger Schema Enforcement

The current app uses manual normalization, not a strict schema validator for LLM output. Future upgrades could use:

- `zod`
- JSON schema validation
- OpenAI structured output support

11.8 Formal Mastra Workflow Layer

`server/src/workflows/interviewFlow.ts` is currently empty. A future workflow graph could formalize:

- question generation
- retries
- evaluation
- analytics
- reference refresh dependencies

---

12. Complete Execution Flow

This section explains the app end to end.

12.1 Login Flow

1. Browser loads `App.tsx`.
2. If auth exists in local storage, the token is restored.
3. Frontend calls `GET /api/auth/me`.
4. Backend verifies the Google token.

5. If valid, the app moves to profile setup.

12.2 Interview Setup Flow

1. User selects target role.
2. User selects years of experience.
3. User selects active skill.
4. User selects difficulty.
5. Frontend calls `POST /api/session`.
6. Backend returns a session id.

12.3 Question Request Flow

1. Frontend calls `POST /api/question`.
2. Backend resolves role context and skill track.
3. Backend loads recent question history for that track.
4. Backend retrieves reference snippets.
5. Backend attempts LLM generation.
6. Backend validates the result.
7. Backend rejects duplicates or weak outputs if necessary.
8. Backend stores question in session memory.
9. Backend returns the final question.

12.4 Theory Answer Flow

1. User types or dictates an answer.
2. Frontend calls `POST /api/answer`.
3. Backend loads the original question from memory.
4. Backend compares answer text with canonical answer keywords.
5. Backend returns:

- score
- feedback
- correction
- next steps

1. Frontend renders the feedback panel.

12.5 Coding Answer Flow

1. User writes code in the code editor.
2. User may click "Run Code", which calls `POST /api/run`.

3. Backend executes code through Judge0 or local VM.
4. User clicks "Evaluate Code".
5. Backend wraps the code with a harness and re-runs it.
6. Harness emits structured `__RESULT__:` output.
7. Backend parses test results and calculates score.
8. Backend returns:

- score
- feedback
- correction
- next steps
- test-by-test results

1. Frontend renders pass/fail cards.

12.6 Next Question Flow

After evaluation, the frontend immediately requests the next question in the background. When the user clicks "Next Question", the app can often show it instantly.

This is a small but effective UX optimization.

---

13. Key Concepts Explained Simply

13.1 LLM

An LLM, or Large Language Model, is a system that predicts the next tokens in text based on patterns learned from large amounts of training data. In this app, the LLM generates interview questions and answer keys.

13.2 Embeddings

Embeddings turn text into vectors so machines can compare meaning, not just exact words. This app does not currently use embeddings. It uses keyword scoring and token overlap instead.

13.3 Prompts

A prompt is the instruction sent to the model. This app uses:

- a system prompt for permanent rules

- a user payload for request-specific context

13.4 Token Usage

Tokens are pieces of text counted by the model API for cost and context length. Long prompts cost more and can slow requests down, so this app compresses context before sending it.

13.5 Temperature

Temperature controls randomness. Higher temperature usually gives more variety but less predictability. Lower temperature gives more consistent outputs. The current implementation uses `0.35`, which leans toward consistency.

13.6 API Requests

An API request is one program asking another for data or work. In this app:

- the frontend calls backend APIs
- the backend calls Google, OpenAI, and Judge0 APIs

13.7 Retrieval

Retrieval means selecting only the most relevant reference snippets instead of sending the whole knowledge base into the prompt.

13.8 Canonical Answer

A canonical answer is the system's best reference answer for a question. It is used to:

- score theory answers
- show corrections
- guide feedback

13.9 Coding Harness

A coding harness is a wrapper around user code that calls the target function with known inputs and captures outputs in a structured way.

13.10 Session State

Session state is the data the backend remembers for a particular interview session, such as:

- asked questions
- recent topics

- recent signatures
- evaluations

Closing Summary

The AI Interview Coach is a well-structured full-stack demo that combines:

- a polished React interview UI
- an Express backend with clear API boundaries
- strict LLM prompt engineering
- deterministic fallback generation
- lightweight keyword-based retrieval
- executable coding evaluation

Its most important architectural strength is that it does not trust the LLM blindly. It wraps generation with:

- controlled prompts
- response parsing
- sanitization
- duplicate suppression
- fallback generation
- explicit evaluation logic

Its main current trade-offs are:

- in-memory storage
- heuristic retrieval instead of embeddings
- simple theory scoring
- partial reference sync support for PDF logic

Even with those trade-offs, the application is a strong example of how to build a practical AI-assisted interview platform that remains understandable to developers and resilient in local development.